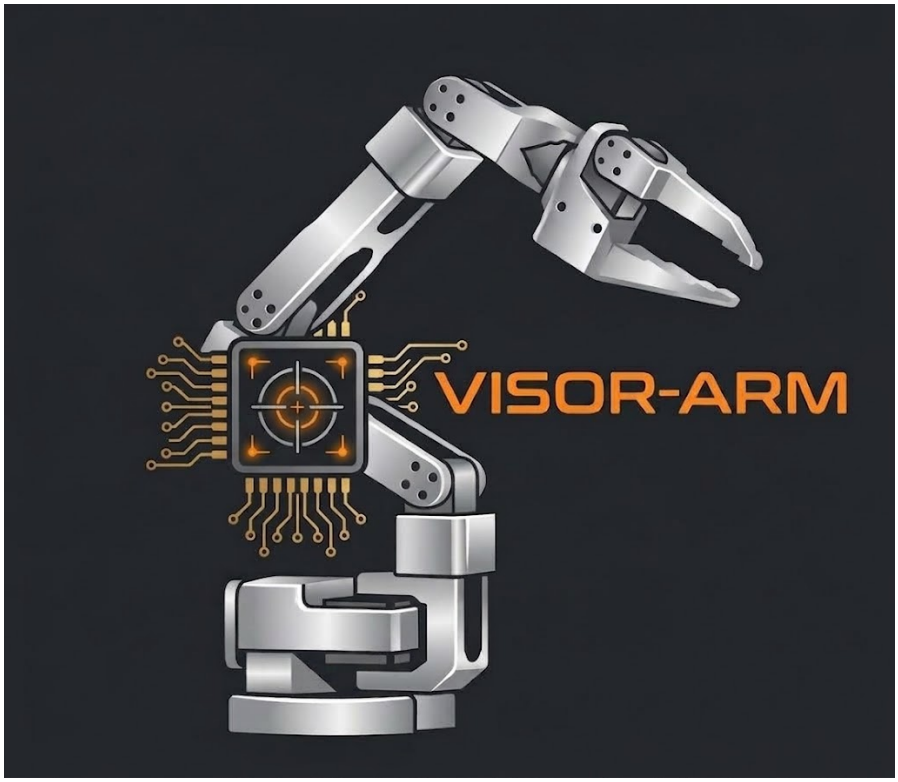
VISOR ARM

AI-Powered Robotic Sorting System

Capstone Project | Technical Documentation | 2026

SO-101 Robotic Arm · Raspberry Pi 5 · YOLO · ACT Imitation Learning · PLC Integration



Project visual from the original VISOR Arm development documentation.

Prepared from the uploaded “VISOR arm steps documentation.docx”. This edited report summarizes the recorded setup, code architecture and observations. It is not a verification of final system performance.

1. Project Overview

VISOR Arm is a capstone robotic sorting project designed to identify plastic bottles and aluminium cans and move them into their corresponding bins. The documented approach combines camera-based object detection with a SO-101 robotic arm and Python-based motion control.

Documented sorting sequence: HOME → DETECTION → YOLO → ACT → CORRECT BIN → RELEASE → DETECTION → REPEAT. A normal stop is designed to finish the current operation and return HOME when no motor fault is present.

2. Hardware and Software

Component	Role in project
SO-101 arm / Feetech servos	Grasping, joint positioning and bin placement
Raspberry Pi 5	Python execution, camera access and robot control
USB camera	640 × 480 capture and object detection
Hailo-8L AI accelerator	Explored for accelerating inference
PLC / conveyor / ultrasonic sensor	Conveyor coordination and object positioning
LeRobot, Python, OpenCV, Ultralytics	Motor control, vision and AI policy tooling

3. Arm Assembly and Calibration

The original notes begin with SO-101 assembly photographs and reference the official LeRobot SO-101 documentation for motor calibration. Subsequent Python examples read Feetech motor positions using the calibration file and the follower serial connection at /dev/ttyACM0.



SO-101 assembly photograph from the source document.

4. Raspberry Pi Camera Setup

The documented setup creates a Python virtual environment, installs OpenCV and NumPy, lists video devices, and uses v4l2-ctl to identify the USB camera. A capture test opens /dev/video0, requests 640 × 480 resolution and saves a test image. The captured test was reported successful in the notes.

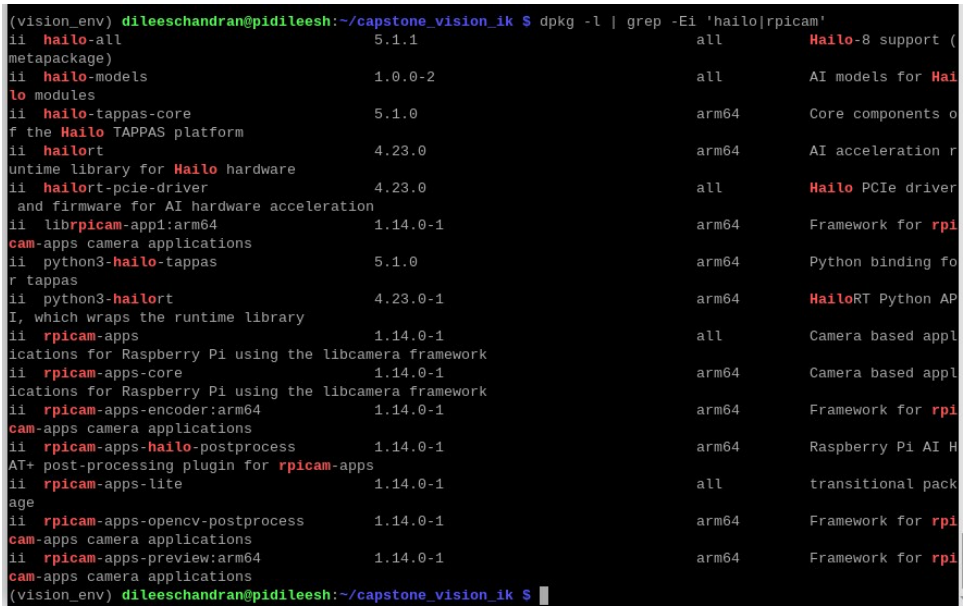
Recorded commands: `ls -l /dev/video*` and `v4l2-ctl --list-devices`. Device numbering should be checked again on each system.

5. Hailo-8L Exploration

The document records a successful Hailo device-identification check and a successful import of the Hailo Python API. A short script loads a Hailo-compatible YOLOv8 HEF file and reports its input and output shapes. This establishes model loading, not a demonstrated end-to-end Hailo inference pipeline.

6. YOLO Object Detection

Ultralytics YOLO was installed to identify two target categories: plastic_bottle and aluminium_can. The source includes screenshots showing a plastic bottle detection at approximately 0.75 confidence and an aluminium can detection at approximately 0.89 confidence. These are example observations, not an overall accuracy benchmark.



One of the source-document screenshots; original figures are retained in the source Word file.

7. Saved Positions and Robot Control

The notes demonstrate reading calibrated joint positions and moving between HOME and DETECTION. A test script moves HOME → DETECTION, runs YOLO, saves a detection image and returns HOME. The report also contains a later continuous-sorting implementation with position feedback, movement verification and explicit handling of faults.

8. ACT-Based Pickup and Bin Placement

The continuous-sorting draft calls a separate ACT rollout command for pickup and transfer, then reconnects the motor bus and selects bottle or can bin poses. The draft preserves the gripper state during transfer, releases the object at the selected bin, and returns to DETECTION for another cycle. ACT model training metrics and a full-cycle success rate are not supplied in this document.

9. Continuous Sorting Software Architecture

The included stage3_auto_sort.py draft imports stage1_detection and stage2_act. It coordinates a camera worker, waits for a recognized object, stops camera access before ACT, runs the ACT subprocess, checks motor feedback, places the object and restarts detection. A Flask dashboard exposes a video feed, robot status and a normal-stop request.

Safety distinction: The source code explicitly labels the dashboard STOP as a normal stop, not an emergency stop. The draft halts automatic HOME recovery after a movement fault. Hardware-level emergency stopping and supervised testing remain separate requirements.

10. Recorded Tests and Evidence

• USB camera capture succeeded at 640 × 480.

• Hailo-8L device identification and Python API checks succeeded.

• Example YOLO bottle and can detections were recorded.

• A HOME → DETECTION → detection → HOME test was documented.

• A continuous sorting Python draft and dashboard code were documented, but the source does not establish validated production reliability.

11. Remaining Work and Documentation Gaps

Before presenting this as a completed autonomous system, add dated results for repeated pickup and placement trials, measured detection-to-action latency, ACT rollout success rates, PLC/MQTT integration testing, gripper design validation, and photos or videos of the full integrated demonstration. Include the actual tested versions of the Python scripts separately in the GitHub repository.

12. Source and References

Primary source: “VISOR arm steps documentation.docx” (59 pages, supplied by the project author). The report retains the source’s terminology and sequence while condensing long code listings into explanations.

LeRobot SO-101 documentation: https://huggingface.co/docs/lerobot/so101

For GitHub publication, review all scripts and screenshots for credentials, private network information and personal details before uploading.